

18. Bölüm

Değişken Argümanlı Fonksiyonlar

18.1 Değişken Sayıda Parametre Tanımı

Belirsiz sayıda argüman alabilen fonksiyona **değişken sayıda argüman alan fonksiyon** (**variadic function**) denir. C dilindeki güçlü ancak çok nadiren kullanılan özelliklerden biridir. C dilindeki `printf()` gibi fonksiyonlardan tanıdığımız klasik kullanımdır.

C Dilindeki gibi C++ dilinde de, belirsiz sayıda bağımsız değişkeni olan bir fonksiyon; **en az bir sabit bağımsız değişkene** sahip olacak şekilde tanımlanır ve ardından derleyicinin değişken sayıda bağımsız değişkeni ayrıştırmasını sağlayan bir üç nokta (...) eklenir. **Birinci parametre kendisinden sonra gelebilecek parametre sayısı hakkında bilgi verir.** Aşağıda buna ilişkin örnek verilmiştir;

```
#include <iostream>
#include <cstdarg>
//ÖNERİLMEZ! TİP GÜVENLİĞİ YOKTUR!
int argumanlarinHepsiniTopla(int kacAdet,...) {
    va_list argumanlar;
    int sayac, toplam = 0;
    va_start(argumanlar, kacAdet);
    for (sayac = 0; sayac < kacAdet; sayac++)
        toplam += va_arg(argumanlar, int);
    va_end(argumanlar);
    return toplam;
}
int main(){
    std::cout << "3 Argüman Toplamı = " << argumanlarinHepsiniTopla(3, 10, 20, 30) << std::endl;
    std::cout << "5 Argüman Toplamı = " << argumanlarinHepsiniTopla(5, 10, 20, 30,40,50) <<
        << std::endl;
}
/*Program Çıktısı:
3 Argüman Toplamı = 60
5 Argüman Toplamı = 150
*/
```

Değişken sayıda parametreleri işlemek için kodunuza `<cstdarg>` başlığını dahil etmeniz gerekir. Bu başlıktaki fonksiyonlar aşağıda verilmiştir;

Fonksiyon	Açıklama
<code>va_start(va_list ap, arg)</code>	Bu fonksiyon, üç nokta ile verilen argümanları <code>va_list</code> değişkenine aktarır.
<code>va_arg(va_list ap, type)</code>	Her seferinde, üç nokta ile temsil edilen değişken listesindeki bir sonraki argümanı <code>va_list</code> üzerinden işler ve listenin sonuna ulaşana kadar onu <code>type</code> ile verilen veri tipine dönüştürür.
<code>va_copy(va_list dest, va_list src)</code> <code>va_end(va_list ap)</code>	<code>va_list</code> 'teki argümanların bir kopyasını oluşturur. Bu, <code>va_list</code> değişkenlerine erişimi sonlandırır.

Tablo 18.1: Stdarg.h Fonksiyonları

<cstdarg> Kullanımı

Modern C++'ta `<cstdarg>` kullanımı **tip güvenliği** (**type safety**) olmadığı için kesinlikle önerilmez!

Bu, günümüz C++ dünyasında en çok tercih edilen yöntem, C++17 ile hayatımıza giren değişken şablonlar (**variadic templates**) ve katlama ifadeleridir (**fold expressions**). Bu konu 15.3 başlığında anlatılmıştır. Tip güvenliği tamdır. Yani sadece `int` değil, her şeyi toplayabilir ve "kaç adet" olduğunu belirtmemize gerek

kalmaz; derleyici bunu senin için hesaplar.

```
#include <iostream>
// Değişken sayıda argüman alan modern şablon fonksiyonu
template<typename... Args>
auto argümanlarınHepsiniTopla(Args... args) {
    // C++17 Fold Expression (Katlama İfadesi): Paketteki tüm elemanları toplar
    return (... + args);
}
int main() {
    // Adet belirtmeye gerek yok!
    std::cout << "3 Arguman Toplami = " << argümanlarınHepsiniTopla(10, 20, 30) << std::endl;
    std::cout << "5 Arguman Toplami = " << argümanlarınHepsiniTopla(10, 20, 30, 40, 50) <<
        std::endl;
}
```

Bir de eski C++11 yöntemi olan liste yöntemi (`std::initializer_list`) bunun için kullanılabilir. Eğer gönderilecek tüm argümanların türü aynıysa (hepsi `int` ise), bu yöntem çok temizdir. Argümanları süslü parantez içinde göndermemiz gerekir.

```
#include <iostream>
#include <initializer_list>
#include <numeric> // std::accumulate için
int argümanlarınHepsiniTopla(std::initializer_list<int> liste) {
    // Listedeki tüm elemanları başlangıç değeri 0 olacak şekilde toplar
    return std::accumulate(liste.begin(), liste.end(), 0);
}
int main() {
    std::cout << "3 Arguman Toplami = " << argümanlarınHepsiniTopla({10, 20, 30}) << std::endl;
    std::cout << "5 Arguman Toplami = " << argümanlarınHepsiniTopla({10, 20, 30, 40, 50}) <<
        std::endl;
}
```

18.2 Değişken Sayıda Argüman Alan Ana Fonksiyon

Ana fonksiyon (main function) programın icra edilmeye başladığı fonksiyondur. Şu ana kadar argüman-sız ana fonksiyonu gördük. C++ Dilinde yazdığımız programlar da çalıştırılırken konsoldan argüman alabilir.

Ana Fonksiyon

- ◊ Satır içi (**inline**) fonksiyon olarak bildirilemez!
- ◊ Adresi alınamaz!
- ◊ Programın başka hiçbir yerinden çağrılmaz (**call**)!

Yazdığımız programlar da çalıştırılırken konsoldan argüman alabilir. Argüman alan ana fonksiyon (**main function**) aşağıdaki gibi iki şekilde tanımlanır;

```
int main(int argc, char* argv[]) {
    // Ana fonksiyon Gövdesi
    // ...
}
//Ya da aşağıdaki şekilde tanımlanır;
int main(int argc, char* argv[], char ** envp) {
    // Ana fonksiyon Gövdesi
    // ...
}
```

Aşağıda konsoldan çalıştırılırken alınan argümanları ve işletim sisteminin ortam değişkenleri (**environment variable**) konsola yazan bir program örneği verilmiştir;

```
#include <iostream>
#include <vector>
#include <string_view>
int main(int argc, char* argv[], char* envp[]) {
    // 1. Argümanları İşleme (Command Line Arguments)
    std::cout << "--- Konsoldan Girilen Argümanlar ---\n";
}
```

```

// argv bir dizidir, modern for döngüsü için bir vektöre kopyalanabilir
// veya doğrudan indislerle erişilebilir.
for (int i = 0; i < argc; ++i) {
    std::cout << "argv[" << i << "]: " << argv[i] << "\n";
}
// 2. Ortam Değişkenlerini İşleme (Environment Variables)
std::cout << "\n--- Ortam (Environment) Degiskenleri ---\n";
// envp, sonu NULL olan bir işaretçi dizisidir.
for (char** env = envp; *env != nullptr; ++env) {
    std::string_view envVar(*env); // Bellek kopyalamadan veriyi oku
    std::cout << envVar << "\n";
}
return 0;
}

```

Programın derlenmesi ve çalıştırılması sonrası çıktı aşağıda verilmiştir;

```

C:\Users\ILHANOZKAN>notepad main.cpp
C:\Users\ILHANOZKAN>gcc main.cpp -o main.exe
C:\Users\ILHANOZKAN>main 10 20 otuz elli

```

Konsoldan Girilen Argümanlar:

```

argv[0]   main.exe
argv[1]   10
argv[2]   20
argv[3]   otuz
argv[4]   elli

```

Ortam Değişkenleri:

```

ALLUSERSPROFILE=C:\ProgramData
APPDATA=C:\Users\ILHANOZKAN\AppData\Roaming
CommonProgramFiles=C:\Program Files\Common Files
CommonProgramFiles(x86)=C:\Program Files (x86)\Common Files
CommonProgramW6432=C:\Program Files\Common Files
COMPUTERNAME=ILHANOZKAN
ComSpec=C:\WINDOWS\system32\cmd.exe
configsetroot=C:\WINDOWS\ConfigSetRoot
DriverData=C:\Windows\System32\Drivers\DriverData
HOMEDRIVE=C:
HOMEPATH=C:\Users\ILHANOZKAN
LOCALAPPDATA=C:\Users\ILHANOZKAN\AppData\Local
LOGONSERVER=\\DAMLANURO
NUMBER_OF_PROCESSORS=8
OneDrive=C:\Users\ILHANOZKAN\OneDrive
OneDriveConsumer=C:\Users\ILHANOZKAN\OneDrive
OS=Windows_NT Path=C:\WINDOWS\system32;C:\Users\ILHANOZKAN\Downloads\codeblocks\MinGW\bin;
PATHEXT=.COM;.EXE;.BAT;.CMD;.VBS;.VBE;.JS;.JSE;.WSF;.WSH;.MSC
PROCESSOR_ARCHITECTURE=AMD64
PROCESSOR_IDENTIFIER=Intel64 Family 6 Model 142 Stepping 12, GenuineIntel
PROCESSOR_LEVEL=6
PROCESSOR_REVISION=8e0c
ProgramData=C:\ProgramData
ProgramFiles=C:\Program Files
ProgramFiles(x86)=C:\Program Files (x86)
ProgramW6432=C:\Program Files
PROMPT=$P$G
PSModulePath=C:\Program Files\WindowsPowerShell\Modules;
PUBLIC=C:\Users\Public
SESSIONNAME=Console
SystemDrive=C:
SystemRoot=C:\WINDOWS
TEMP=C:\Users\ILHANOZKAN\AppData\Local\Temp

```

```
TMP=C:\Users\ILHANOZKAN\AppData\Local\Temp
USERDOMAIN=DAMLANURO
USERDOMAIN_ROAMINGPROFILE=DAMLANURO
USERNAME=ILHANOZKAN
USERPROFILE=C:\Users\ILHANOZKAN
windir=C:\WINDOWS
ZES_ENABLE_SYSMAN=1

C:\Users\ILHANOZKAN>
```